

Appendix S1. Data simulation and analysis for the FP-FN occupancy model

Ismael V. Brack Denis Valle

April 26, 2026

Table of contents

Required Packages	1
1. Data simulation	2
Simulation settings	2
Simulate occupancy and availability states	3
Simulate data collection and automated detections	4
Simulate object-level data	4
2. Data analysis	7
Specify model for JAGS	7
Organize data for JAGS	8
Fit the FP-FN occupancy model	9

This Appendix is a supporting document for the manuscript “*Estimating wildlife occupancy and abundance using automated object detections in images*” (Brack et al., 2026). First, we show the data simulation process for generating the object detections in camera trap images, containing true-positive (TP) and false-positive (FP) detections and their associated confidence scores. Then, we analyze the resulting data with our FP-FN occupancy model (single-species, single-season). The model relies on a verification procedure applied to a subset of images in which the objects are classified into TP and FP and inform the scores distribution of each class.

Our model structure is similar to other previous occupancy models that account for FP and FN errors in automated detections (Chambert et al. 2018; section 7.6 of Kéry & Royle 2021), but with a major difference that we model the class and number of objects within each image/file based on their scores. Another difference is that we include an intermediate level between occupancy and detection to model the TP rates conditioned on the species availability in the occasion, rather than modeling TP and FP rates per site.

Required Packages

We will need the following packages to run this code:

```
library(jagsUI)
library(ggplot2)
library(MCMCvis)
```

Note that you also need to have installed JAGS software in your PC. You can download it [here](#).

1. Data simulation

In this example, we will simulate 50 camera-trap sites that are surveyed for 15 sequential occasions (e.g., 15 days). We will consider that a camera takes on average 6 photos per occasion. For simulating the occupancy probability, availability probability, TP rate and FP rate, we will add some heterogeneity in these parameters. Specifically, we assume that the occupancy probability has a positive relationship with a habitat covariate (e.g., forest); the availability probability has a positive relationship with a temporal covariate (e.g., temperature); the TP rate has a negative relationship with the same habitat covariate as occupancy; and we will include a site-level random effect for the FP rates. We first simulate the TP and FP detections of each image, at each occasion of each site, conditioned on the species occupancy and availability, to then simulate the object-level information containing the true class and confidence score.

Simulation settings

First, we define the true values for the parameters and define the simulation settings for the sampling design:

```
nsites = 50 # number of sites
nvisits = 15 # number of occasions / visits

mean.img = 6 # average number of photos per occasion per site

# Proportion of images with detections that are verified
prop.verif <- 0.1

# Simulate covariates (centered)
forest <- runif(nsites,-3,3)
temperature <- matrix(runif(nsites*nvisits,-3,3) ,nsites,nvisits)

# Occupancy prob.
mean.psi = 0.5
alpha0 = qlogis(mean.psi) # intercept
alpha1 = +1 # slope

psis = plogis(alpha0 + alpha1*forest) # occupancy per site

# Availability prob.
mean.phi = 0.4
beta0 = qlogis(mean.phi) # intercept
beta1 = +0.4 # slope
```

```

phis <- plogis(beta0 + beta1*temperature) # availability per site-occasion

# TP rates: true positives per images given that the species is available
mean.lam <- 0.5
delta0 <- log(mean.lam) # intercept
delta1 <- -0.3 # slope

lambdas <- exp(delta0 + delta1*forest) # tp rate per site

# FP rates: number of detected FPs per image
mean.ome <- 0.4
mu.ome <- log(mean.ome) # intercept
sd.ome <- 0.4 # slope

omegas <- exp(rnorm(nsites, mu.ome, sd.ome)) # fp rate per site

# Confidence scores
tp.mu = qlogis(.75) # mean score for TPs
fp.mu = qlogis(0.5) # mean score for FPs

sd.scr = 0.5 # sd for scores

```

Simulate occupancy and availability states

Next, we simulate the occupancy state in the sites and the availability state at each occasion of each site:

```

# Occupancy state
z <- rbinom(nsites,1,psis)

# Empty
u <- matrix(NA, nsites, nvisits)

# Availability process
for(s in 1:nsites){
  # True positives process
  # Species availability in the site for a visit
  u[s,] <- rbinom(nvisits,1,z[s]*phis[s,])
}

```

Let's take a look at the occupancy state "z" and the availability state "u" of the first 10 sites:

	z															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	1	1	1	0	1	1	0	0	1	0	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0
1	1	1	1	0	0	0	1	1	0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

z															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	0	0	1	0	1	1	1	0	1	1	1
1	0	1	1	0	0	0	0	0	1	1	1	0	0	0	0
1	0	0	1	0	0	1	1	1	1	1	0	0	1	1	1

Simulate data collection and automated detections

We first simulate the realized number of photos that is taken at each day at each site using a Poisson distribution. Then, we simulate the number of TP and FP objects in each of these images. This process results in a three-dimensional array (sites x occasions x images) for both TPs and FPs. However, as we do not verify each image, the resulting data is only the total number of detections per image.

```
## Simulate data collection

# Number of images at each occasion of each site
nimgs <- matrix(rpois(nsites*nvisits, mean.img),nsites,nvisits)

# Empty objects
TPs <- FPs <- array(NA, dim=c(nsites, nvisits, max(nimgs)))

for(s in 1:nsites){
  for(j in 1:nvisits){
    # jump loop if there is no image taken in this visit
    if(nimgs[s,j]==0){next}

    # Number of TP detected objects in the image
    TPs[s,j,1:nimgs[s,j]] <- rpois(nimgs[s,j],lambdas[s] * u[s,j])

    ## False positives process
    FPs[s,j,1:nimgs[s,j]] <- rpois(nimgs[s,j], omegas[s])
  } #j
} #s

ndets <- TPs + FPs # total number of detections per image (3d array)
```

Note that, because different visits can have different number of images collected, we have to create the array using the maximum number of images taken in a visit, and keep ‘NA’ when the image does not exist.

Simulate object-level data

Now, we use the TP and FP detections to extract the class of each object at each image and simulate their confidence scores. Object scores are simulated according to its true class (TP or FP) distribution.

```

## Simulate object-level data
obj.data <- list()

oo=0
for(s in 1:nsites){
  for(j in 1:nvisits){

    # jump loop if there is no image taken in this visit
    if(nimgs[s,j]==0){next}

    for(i in 1:nimgs[s,j]){
      oo=oo+1
      # Only if there has been a detection in this image
      if(ndets[s,j,i]>0){
        # organize temporary table
        tab <- data.frame(
          # True/False detection
          class=c(rep(1,TPs[s,j,i]),rep(0,FPs[s,j,i])),
          score=NA,
          site=s,
          visit=j,
          image=i
        )
        # Scores for each detected object
        # TP scores
        if(TPs[s,j,i]>0){
          tab[which(tab$class==1),"score"] <-
            rnorm(TPs[s,j,i], tp.mu, sd.scr)
        }
        # FP scores
        if(FPs[s,j,i]>0){
          tab[which(tab$class==0),"score"] <-
            rnorm(FPs[s,j,i], fp.mu, sd.scr)
        }

        obj.data[[oo]] <- tab

      } # ndets>0
    } #i
  } #j
} #s

obj.data <- do.call(rbind, obj.data) # transform to data frame

```

Verification procedure

Finally, we filter a random sample of the images that contain at least one detection to be verified. To do this, we first need to organize a table containing information about the images from which we will sample the verified ones. We assume that if an image is selected, all the objects within it are verified. Additionally, we have to transform the proportion of images reviewed into a number.

```

## Sample images to verify and observe classes

# Create data.frame containing the site, visit, image and ndets
images <- as.data.frame(table(obj.data[,c("site","visit","image")]),
                        responseName="ndets")

# Exclude site-visits with no images or no detected objects
no.obj <- which(is.na(ndets), arr.ind=T)
images <- images[which(!paste(images$site,images$visit,images$image) %in%
                             paste(no.obj[,1], no.obj[,2], no.obj[,3])),]

# Sample images with at least one detection to verify
nverif <- round(sum(images$ndets>0)*prop.verif) # number of verified images
# position of the sampled images
verifs <- sample((1:nrow(images))[which(images$ndets>0)],nverif)

# Keep only verified images
images <- images[verifs,]

# add a column for the observed classes
obj.data$obs.class <- NA

# positions of all objects verified
aqui <- which(paste0(obj.data$site, obj.data$visit, obj.data$image) %in%
              paste0(images$site, images$visit, images$image))

# Fill the observed class with the true class
obj.data[aqui,"obs.class"] <- obj.data[aqui,"class"]

```

This is how the object-level data looks like:

	class	score	site	visit	image	obs.class
	0	-0.5110008	25	12	4	NA
	0	0.2860102	24	2	1	NA
	1	0.9248271	5	2	5	NA
	0	-0.6851440	36	13	1	NA
	0	-0.0004505	7	14	1	NA
	0	-0.7094740	11	10	1	NA
	0	-0.3498995	34	4	3	NA
	0	-0.4910028	36	14	6	0
	0	0.7231152	37	3	2	0
	0	-1.0923062	41	14	2	0
	1	1.0783495	37	5	2	1
	0	0.8222868	10	5	2	0
	1	0.5423723	4	9	1	1
	0	0.4915113	43	12	2	0
	0	-0.7164222	14	8	2	0

2. Data analysis

For analyzing the data, we will use the two objects created above: the 3D array containing the total number of detections in each image, at each occasion of each site, and the object-level data.

Specify model for JAGS

We specify the JAGS model using the code below. We model the number of detections (3D array) as a resulting process from occupancy, availability, and TP/FP rates, and connect that with the object-level data using the expected proportion of TPs.

```
occu.model <- "
model{

  ### psi(forest) phi(temp) lam(forest) ome(site.raneff)

# Priors -----
# Occupancy probability (psi)
alpha0 ~ dnorm(0,0.01) # intercept
alpha1 ~ dnorm(0,0.01) # slope

# Availability prob. (phi)
beta0 ~ dnorm(0,0.01) # intercept
beta1 ~ dnorm(0,0.01) # slope

# TP rate (lambda)
delta0 ~ dnorm(0,0.01)
delta1 ~ dnorm(0,0.01)

# FP rate (omega)
mu.ome ~ dnorm(0,0.01)
sd.ome ~ dunif(0,10)
tau.ome <- 1 / sd.ome^2

# Scores
mu.scr[1] ~ dnorm(0, 0.01) # mean score values for class=0
mu.dif ~ dunif(0,5)      # impose restriction: mu2 >= mu1
mu.scr[2] <- mu.scr[1] + mu.dif # mean score value for class=1

sd.scr ~ dunif(0, 10) # std.dev. for TP and FP scores
tau.scr <- 1 / sd.scr^2

# Likelihood -----
for(s in 1:nsites){

  # linear relationship for occupancy prob.
  logit(psi[s]) <- alpha0 + alpha1*forest[s]

  z[s] ~ dbern(psi[s]) # latent occupancy state
```

```

# linear relationship for TP rates
log(lambda[s]) <- delta0 + delta1*forest[s]

# random effects for precision
gamma0[s] ~ dnorm(mu.ome, tau.ome)
log(omega[s]) <- gamma0[s]

for(j in 1:nvisits){

  u[s,j] ~ dbern(z[s]*phi[s,j]) # species available in occasion/visit

  # linear relationship for availability prob.
  logit(phi[s,j]) <- beta0 + beta1*temperature[s,j]

  for(i in 1:nimgs[s,j]){

    # total number of objects detected per image
    ndets[s,j,i] ~ dpois(lambda[s] * u[s,j] + omega[s])

    # Proportion of TPs (aka precision)
    tp.prop[s,j,i] <- lambda[s]*u[s,j] /
                      (lambda[s]*u[s,j] + omega[s])

  } #i
} #j
} #s

# Object data
for(k in 1:nobj){
  # true latent class (TP of FP)
  class[k] ~ dbern(tp.prop[site[k],visit[k],image[k]])
  # object-level scores
  score[k] ~ dnorm(mu.scr[(class[k]+1)], tau.scr)
} #k
} "

# Save model code as a temporary .txt file
temp.mod1 <- tempfile(fileext = ".txt")
writeLines(occu.model, temp.mod1)

```

Note that, different from some previously developed approaches, we restrict the mean of TP scores to be equal or higher the mean of FPs to improve model convergence. We believe this is a very reasonable assumption for most AI trained models.

Organize data for JAGS

We start by bundling the data required to run the JAGS model:

```
# Bundle data
dat <- list(
  nsites=nsites,nvisits=nvisits,nimgs=nimgs,
  # covariates
  forest=forest,temperature=temperature,
  # 3d array with detections
  ndets=ndets,
  # object data
  class=obj.data$obs.class,score=obj.data$score,
  site=obj.data$site,visit=obj.data$visit,image=obj.data$image,
  nobj=nrow(obj.data)
)
```

Initial values for the MCMC algorithm:

To avoid issues with initial values, we define all the unobserved latent states as one.

```
# Initial values
inits <- function() list(
  z=rep(1,nsites), u=matrix(1,nsites,nvisits),
  class=ifelse(is.na(obj.data$obs.class),1,NA)
)
```

Next, we define the parameters that we want to be monitored in MCMC and define the MCMC settings.

```
# Parameters monitored
params <- c("alpha0","alpha1", "beta0","beta1",
  'delta0','delta1', "mu.ome","sd.ome",
  "mu.scr","sd.scr",
  "psi","phi","lambda","omega"
)

# MCMC settings
ni <- 80000 # number of iterations
nt <- 2 # thinning rate
nb <- 40000 # burn-in
nc <- 3 # number of chains
```

Fit the FP-FN occupancy model

Finally, we run the occupancy model (note it can take several minutes to run):

```
out <- jags(dat, inits, params,
  model.file=temp.mod1,
  n.chains=nc,n.thin=nt, n.iter=ni, n.burnin=nb,
  parallel=T)
```

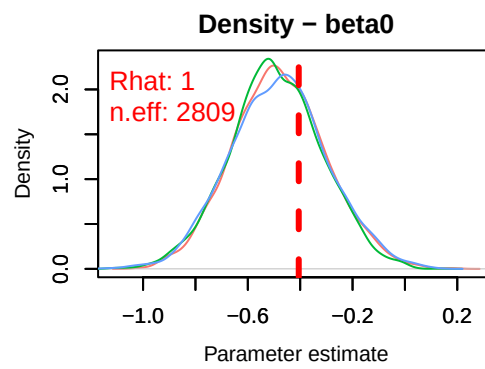
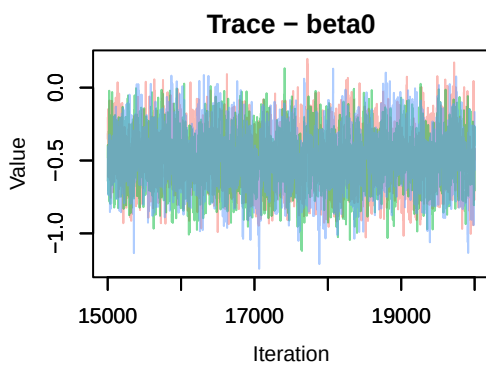
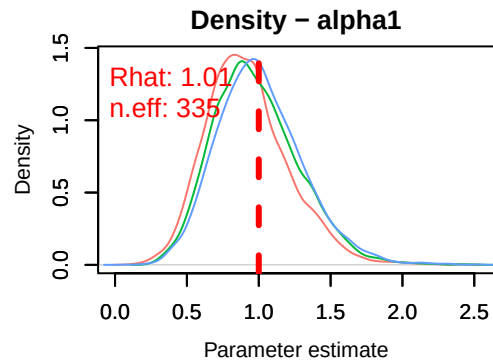
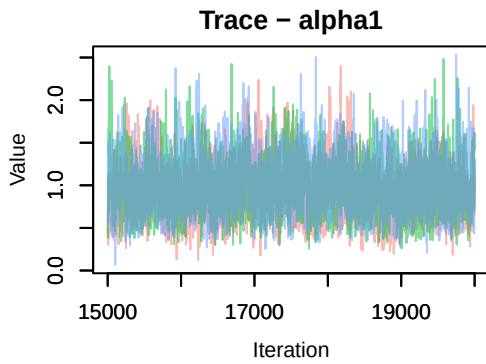
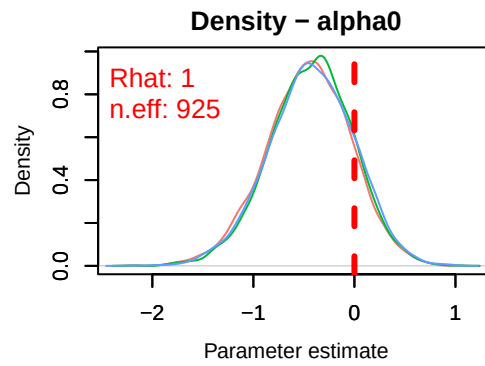
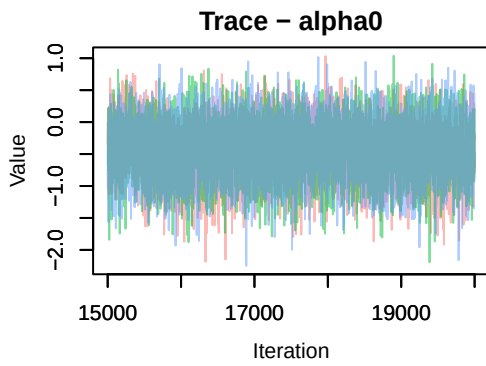
Let's see the summarized posterior samples to examine the model results:

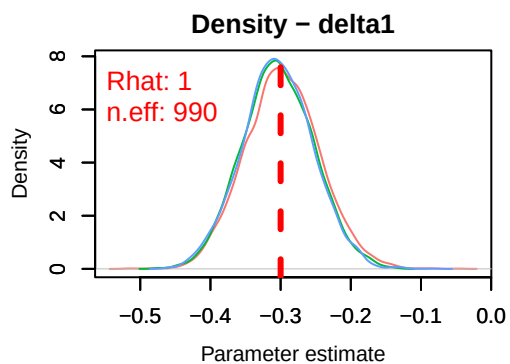
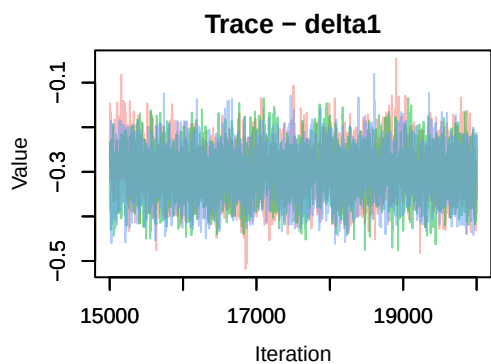
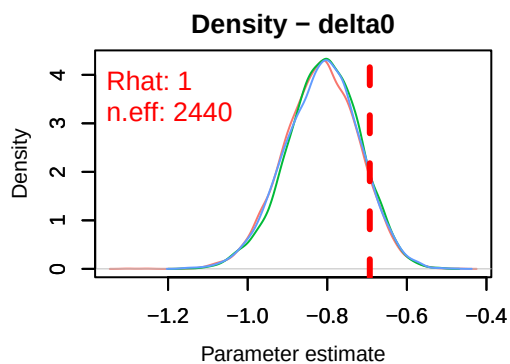
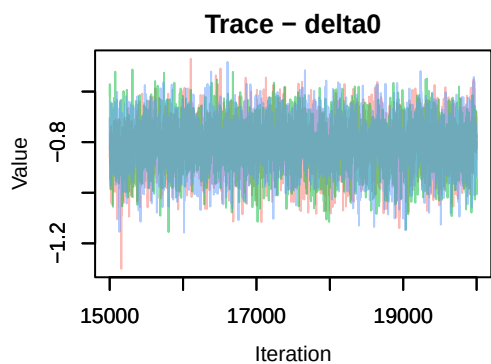
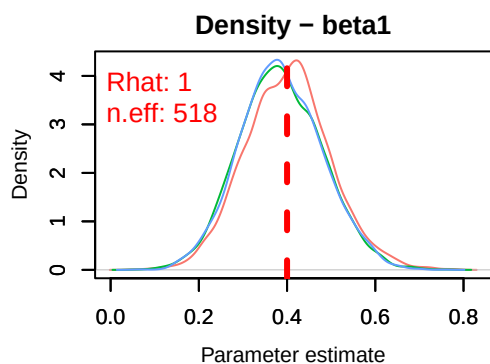
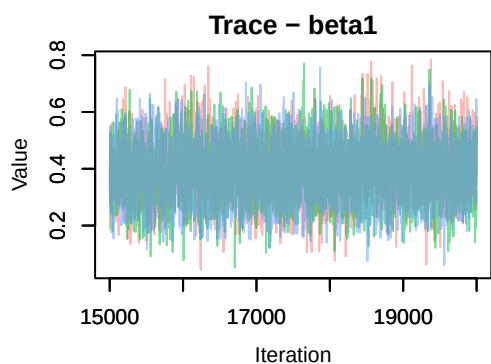
```
print(round(out$summary[c(1:10),-11],3))
```

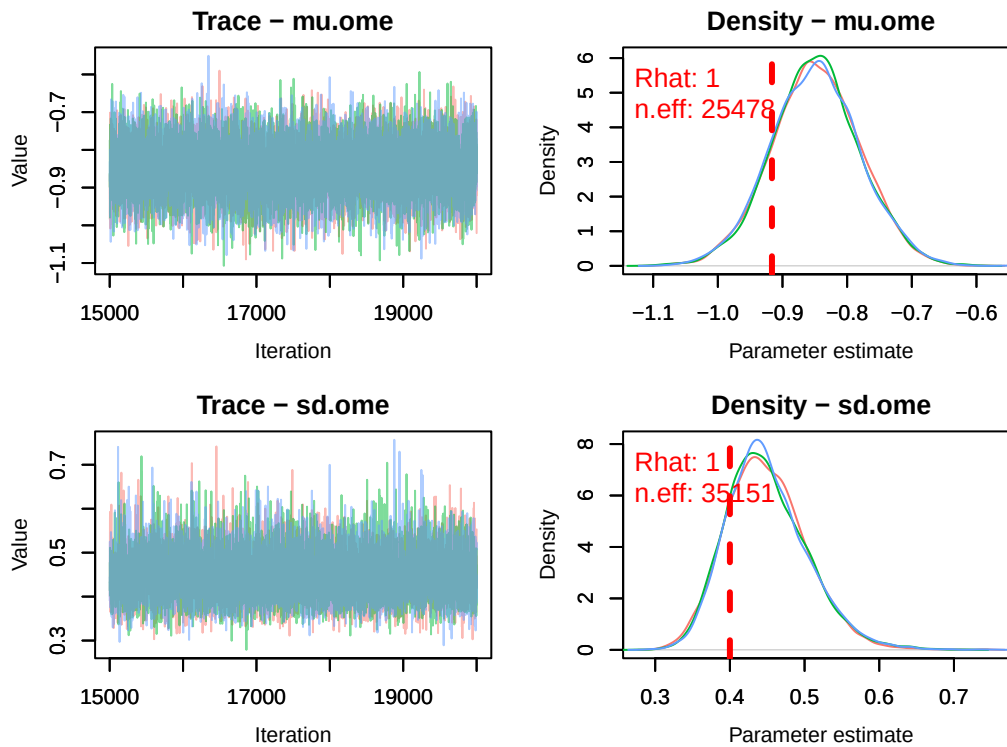
	mean	sd	2.5%	25%	50%	75%	97.5%	Rhat	n.eff	overlap0
alpha0	-0.454	0.418	-1.311	-0.727	-0.444	-0.169	0.332	1.002	925	1
alpha1	0.955	0.292	0.451	0.751	0.929	1.130	1.600	1.007	335	0
beta0	-0.484	0.181	-0.835	-0.606	-0.484	-0.364	-0.127	1.001	2809	0
beta1	0.395	0.095	0.214	0.330	0.393	0.458	0.587	1.004	518	0
delta0	-0.815	0.095	-1.008	-0.877	-0.812	-0.750	-0.637	1.001	2440	0
delta1	-0.299	0.052	-0.399	-0.334	-0.299	-0.264	-0.195	1.002	990	0
mu.ome	-0.847	0.068	-0.983	-0.892	-0.846	-0.801	-0.713	1.000	25478	0
sd.ome	0.449	0.054	0.355	0.411	0.445	0.483	0.568	1.000	35151	0
mu.scr[1]	-0.008	0.012	-0.031	-0.016	-0.008	0.000	0.015	1.002	1201	1
mu.scr[2]	1.146	0.039	1.070	1.120	1.146	1.172	1.223	1.001	1769	0

We can also see the traceplots and posterior distributions for the monitored parameters. Note that each chain is represented by one color and the vertical red line represent the true parameter value used to simulate data.

```
# traceplots..
MCMCtrace(out,ind=T,pdf=F,
           Rhat=T,n.eff=T,
           params=c("alpha0","alpha1","beta0","beta1",'delta0','delta1',
                    "mu.ome","sd.ome"),
           gvals=c(alpha0,alpha1,beta0,beta1,delta0,delta1,mu.ome,sd.ome))
```





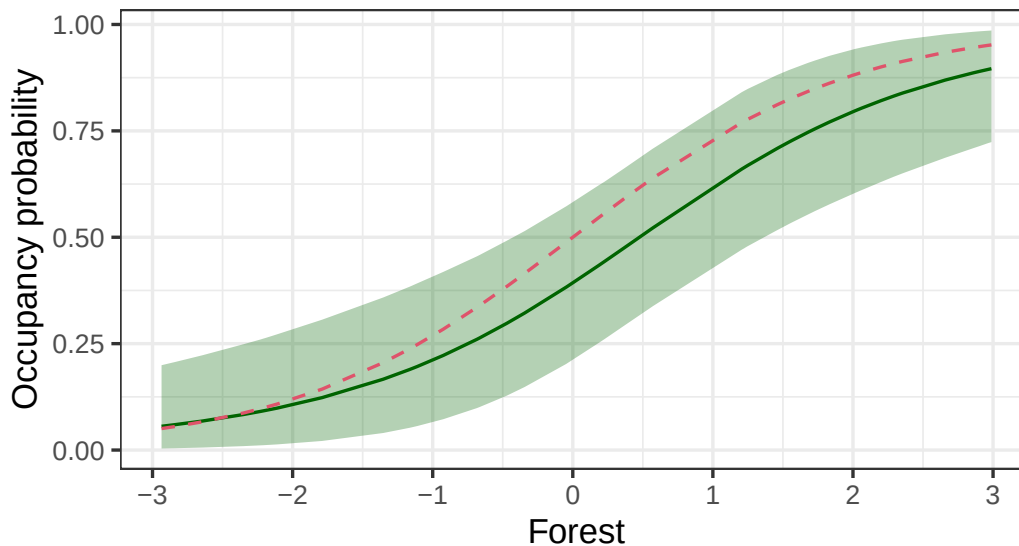


Parameter estimates look good! Estimated values are close to the true values used to simulate the data. It is important to check MCMC convergence in the results (e.g., `Rhat<1.1`).

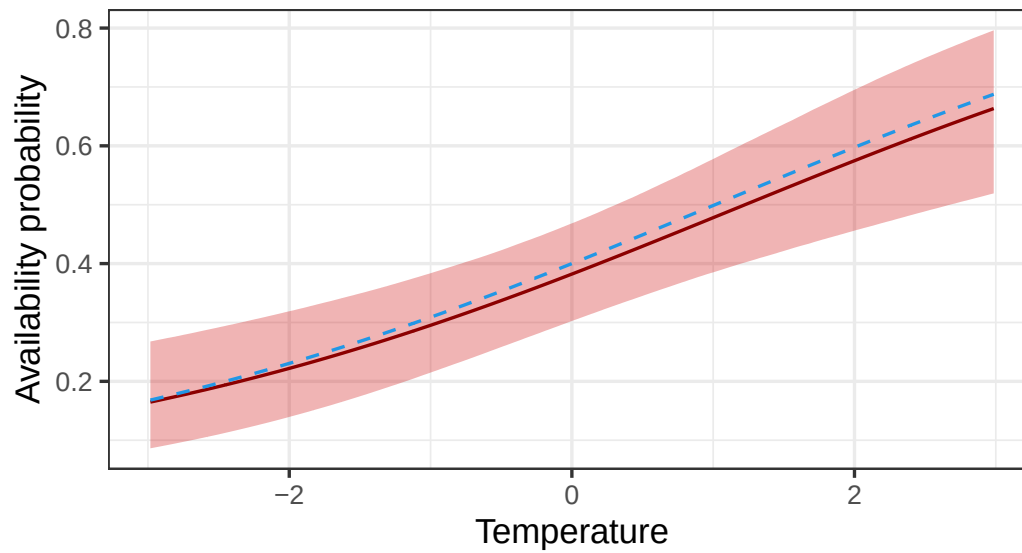
Now, we take a look at the estimated relationships with the covariates for the different probabilities. For this, we first organize the results for plotting the estimates with the 95% credible intervals and compare with the true relationship line used to simulate data (dashed lines):

```
##* Occupancy
psi.resu <- data.frame(
  true = psis,
  mean = out$mean$psi,
  lcl = out$q2.5$psi,
  ucl = out$q97.5$psi,
  forest=forest
)
ggplot(data=psi.resu, aes(x=forest,y=mean,ymin=lcl,ymax=ucl)) +
  geom_ribbon(fill="darkgreen",alpha=.3) +
  geom_line(col="darkgreen") +
```

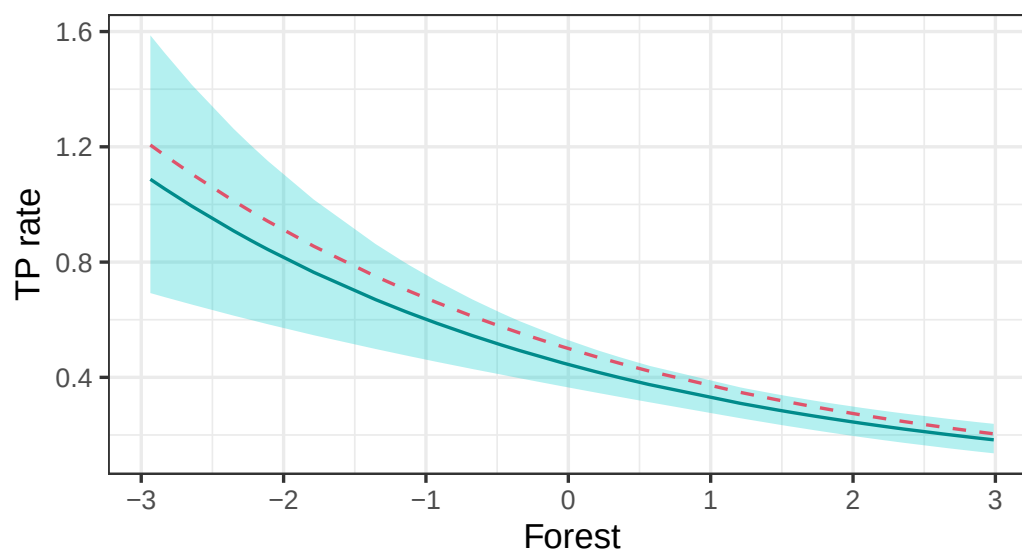
```
labs(y="Occupancy probability",x="Forest") +
geom_line(aes(x=forest,y=true),col=2,linetype="dashed") +
theme_bw(base_size=13)
```



```
## Availability
phi.resu <- data.frame(
  true = as.vector(phis),
  mean = as.vector(out$mean$phi),
  lcl = as.vector(out$q2.5$phi),
  ucl = as.vector(out$q97.5$phi),
  temperature= as.vector(temperature)
)
ggplot(data=phi.resu, aes(x=temperature,y=mean,ymin=lcl,ymax=ucl)) +
  geom_ribbon(fill="red3",alpha=.3) +
  geom_line(col="red4") +
  labs(y="Availability probability",x="Temperature") +
  geom_line(aes(x=temperature,y=true),col=4,linetype="dashed") +
  theme_bw(base_size=13)
```



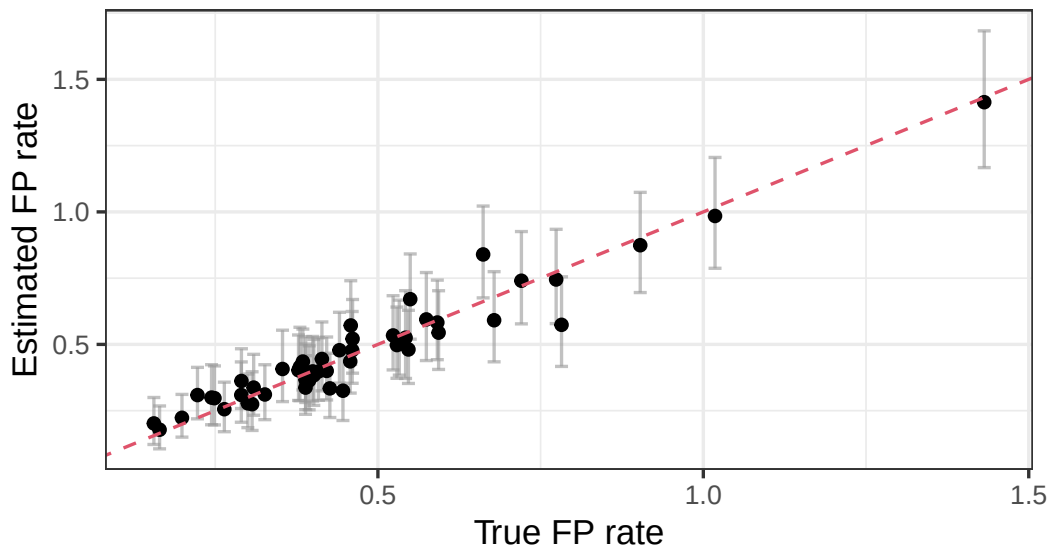
```
## TP rates
tp.resu <- data.frame(
  true = lambdas,
  mean = out$mean$lambda,
  lcl = out$q2.5$lambda,
  ucl = out$q97.5$lambda,
  forest=forest
)
ggplot(data=tp.resu, aes(x=forest,y=mean,ymin=lcl,ymax=ucl)) +
  geom_ribbon(fill="cyan3",alpha=.3) +
  geom_line(col="cyan4") +
  labs(y="TP rate",x="Forest") +
  geom_line(aes(x=forest,y=true),col=2,linetype="dashed") +
  theme_bw(base_size=13)
```



We can also check the relationship between the estimated FP rates for the different sites (that come from

a site-level random effect) and the true FP rate used to simulate data:

```
fp.resu <- data.frame(  
  true = omegas,  
  mean = out$mean$omega,  
  lcl = out$q2.5$omega,  
  ucl = out$q97.5$omega  
)  
  
ggplot(data=fp.resu, aes(x=true,y=mean,ymin=lcl,ymax=ucl)) +  
  geom_errorbar(width=.02,col="gray60",alpha=0.6) +  
  geom_point() +  
  labs(y="Estimated FP rate",x="True FP rate") +  
  geom_abline(intercept=0,slope=1,col=2,linetype="dashed") +  
  theme_bw(base_size=13)
```



Figures are showing that our FP-FN occupancy model is accurately rescuing the parameter values and the relationships with the covariates used to simulate the data!